Ch 09: Sets

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Sets

Set

A set contains a collection of unique values and works like a mathematical set.

- **Set**: object that stores a collection of unique data items
  - All items must be unique
  - Set is unordered
  - Elements can be of different data types

Format for creating a set:

- `argument` can be a list, string or tuple (a sequence)
- if `argument` is a string, each character becomes a set element

```
emptyset = set()
myset = set(['a', 'b', 'c', 'a', 'd'])
print(myset)
{'d', 'b', 'c', 'a'}
```



EAST TEXAS A&M

Getting the Number of and Adding and Deleting Elements of Sets

- `len` function: returns the number of elements in set
- Sets are mutable objects
- `add()` method: adds an element to a set
- `update()` method adds a group of elements to a set
  - Arguments must be a sequence containing iterable elements, and each of the elements is added to set
- `remove()` and `discard()` methods: remove the specified item from the set
  - The item that should be removed is passed to both as argument
  - Behave differently when specified item is not found
    - `remove()` method raises `KeyError` exception if not found
    - `discard()` method just quietly ignores request to discard element not in set

```
myset = set()
myset.add(1)
myset.add(2)
set2 = set([2, 4, 6])
myset.update(set2)
print(myset)
print(len(myset))
myset.remove(2)
print(myset)
{1, 2, 4, 6}
4
{1, 4, 6}
```

EAST TEXAS A&M

Using the `for` Loop `in` and `not in` Operators With a Set

- A `for` loop can be used to iterate over elements in a set
  - General format: `for item in set:`
  - The loop iterates once for each element in the set

```
myset = set(['a', 'b', 'c'])
for val in myset:
    print(val)
b
c
a
```

- The `in` operator can be used to test whether a value exists in a set
- Similarly the `not in` operator can be used to test whether a value does not exist in a set

```
myset = set([1, 2, 3])
if 1 in myset:
    print('The value 1 is in the set.')

if 4 not in myset:
    print('The value 4 is not in the se
The value 1 is in the set.
The value 4 is not in the set.
```

EAST TEXAS A&M

Set Union, Intersection and Difference

Union: a set that contains all the elements of both sets

- Use either the `union()` member method or the `|` operator
- Both return a new set which contains union

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.union(set2)
set4 = set2 | set1
print(set3)
print(set4)
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
```

Intersection: a set that contains only the elements found in both sets

- Use either the `intersection()` member method or the `&` operator
- Both return a new set which contains the intersection

```
set3 = set1.intersection(set2)
set4 = set2 & set1
print(set3)
print(set4)
{3, 4}
{3, 4}
```

Difference: a set that contains the elements that appear in the first set but do not appear in the second set

- Use either the `difference()` member method or the `-` operator
  - **Note** unlike union and intersection, difference is not symmetrical

```
set3 = set1.difference(set
set4 = set2 - set1
print(set3)
print(set4)
{1, 2}
{5, 6}
```



Symmetric Difference and Subsets and Supersets

Symmetric difference: a set that contains the elements that are not shared by the two sets

- Use either the `symmetric_difference()` member method or the ^ operator

Subset: Set A is a subset of set B if all the elements in set A are included in set B

- Use either the `issubset()` member method or the <= operator

Superset: Set A is a superset of set B if it contains all of the elements of set B

- Use either the `issuperset()` member method or the >= operator

EAST TEXAS A&M

Set Comprehensions

- **Set comprehension**: a concise expression that creates a new set by iterating over the elements of a sequence
  - Like list and dictionary comprehensions, except a set comprehension is enclosed in curly braces {}
    - Difference in syntax is don't specify `key: value` in comprehension like a dictionary comprehension

Example make a copy of a set

```
set1 = set([1, 2, 3, 4, 5])
set2 = {item for item in set1}
print(set2)
{1, 2, 3, 4, 5}
```

Example create a set that contains the squares of the numbers in another set

```
set1 = set([1, 2, 3, 4, 5])
set2 = {item**2 for item in set1}
print(set2)
{1, 4, 9, 16, 25}
```



Summary

- A set contains a collection of unique values and works like a mathematical set
  - Create a set using the built-in `set()` method
  - Use `add()` `update()` `remove()` `discard()` to add and remove items from sets
  - Sets support union, intersection, difference, symmetric difference subsets and supersets, just like a mathematical set
  - Set comprehensions work similar to list and dictionary comprehensions



EAST TEXAS A&M

References



EAST TEXAS A&M